Student name

Vignesh V Hegde

SRN / Roll number

R23EJ138

Program and Section

CSIT 'C'

Project title

Curriculum Design Framework

Faculty guide

Prof. Kavya A K

Major Project Report — Phase 1 Review

Curriculum analysis, design, implementation, testing and phase-two plan

Report basis

This report summarizes the project repository, requirements, README, LLDs and test inventory. Personal experience and test-run outcomes are not inferred. Any items requiring personal records are clearly identified for completion.

Executive summary

The Curriculum Design Framework is an academic decision-support platform for curriculum construction and verification. Its documented scope includes curriculum and version management, outcomes and mappings, prerequisites, document ingestion, graph projection and deterministic verification. Findings are advisory and require human review; the system does not approve curricula or claim official accreditation certification.

1. Literature survey and requirement analysis

1.1 Problem statement

Faculty and curriculum committees must review interconnected courses, semester sequencing, credits, course outcomes (COs), program outcomes (POs), PEOs, prerequisites and content. Manual review can miss gaps and inconsistencies across a complete program. This project organizes curriculum information and applies repeatable checks to highlight evidence-backed issues for academic review, supporting—not replacing—expert judgment.

1.2 Literature survey

Source	Approach	Strength	Limitation / relevance
CDIO, “CDIO Curriculum Design for Computing: A Graph-Based Approach” (CDIO proceedings; source PDF: cdio.org/files/document/file/81.pdf)	Graph model of courses, learning goals/topics and prerequisite dependencies.	Supports graph-based representation and curriculum relationship analysis.	Graph structure alone does not deliver versioned governance, workflow or findings review.
“Course-prerequisite networks for analyzing and understanding academic curricula,” Applied Network Science (2023), doi:10.1007/s41109-023-00543-w	Directed course networks used to analyze prerequisite relationships and curriculum structure.	Relevant to sequencing, connectivity and visualization.	Prerequisite analysis alone does not cover outcome mapping, documents or institutional targets.
“A scoping review of methods used to analyze engineering curricula quantitatively with curricular analytics” (review; hksmp.com/journals/eer/article/view/1110)	Surveys quantitative curricular analytics and network methods.	Highlights diverse metrics and need for context-specific evaluation.	Data definitions and evaluation settings vary; results require local expert validation.
Anderson & Krathwohl (eds.), A Taxonomy for Learning, Teaching, and Assessing (2001), LC record loc.gov/item/00063423	Revised Bloom taxonomy categorizes cognitive process and knowledge dimensions.	Useful vocabulary for interpreting learning outcomes and progression.	Taxonomy does not establish correctness of a particular curriculum mapping.

Bibliographic note: confirm complete author lists, publication details and citation format from source publications before formal submission.

1.3 Literature gaps and project response

Gap 1: course-by-course reviews may miss relationships across semesters and outcomes. **Gap 2:** automated flags can be opaque and confuse redundancy with legitimate progression. **Gap 3:** fixed accreditation or credit assumptions may not reflect institutional rules. The project responds with version-scoped analysis, stable evidence, explicit recommendations and configurable governed references, with faculty review retained.

1.4 Requirements

ID	Requirement	Type	Priority
R1	Secure user authentication and role-based curriculum access.	Functional	High
R2	Manage curricula, semesters, courses and version history.	Functional	High
R3	Manage COs, POs, PEOs, CO–PO mapping and review history.	Functional	High
R4	Upload curriculum documents, validate and extract text/analysis.	Functional	Medium
R5	Detect invalid, cyclic, out-of-order and potentially missing prerequisite links.	Functional	High
R6	Run structural, outcome, content and reference checks; produce evidence-backed reports.	Functional	High
R7	Preserve data integrity, explainability, stale-run awareness and mandatory human review.	Non-functional / governance	High

2. System and detailed design

2.1 System design

The Next.js App Router frontend calls authenticated FastAPI REST endpoints. Routers delegate business logic to Python services; repositories isolate SQLAlchemy persistence. PostgreSQL is the authoritative store for curriculum data, versions, authorization and audit records. Neo4j is an optional, rebuildable, version-scoped projection for relationship exploration. Document processing uses storage/extraction adapters and LLM assistance for extraction/drafting, not verification verdicts. A durable projection-job worker refreshes graph data after relational commits.

```
Faculty / curriculum committee
        ↓
Next.js workspace → FastAPI REST (JWT)
        ↓
Services → Repositories → PostgreSQL (source of truth)
   ■■ Document extraction / LLM-assisted drafting
   ■■ Deterministic verification → findings → human review
   ■■ Projection jobs → Neo4j (optional)
```

2.2 Detailed design diagrams

Diagram	Key content	Code / system mapping
Flowchart	Upload → validate → extract → persist; verify version → checks → findings → review.	Document and verification APIs/services.
UML sequence/class	Router, service, repository, model; external storage/LLM adapters; worker interaction.	backend/app/api, services, repositories, schemas and db/models.
DFD Level 0	Faculty, application, document/LLM providers, PostgreSQL, optional Neo4j.	REST interfaces and infrastructure dependencies.
DFD Level 1	Curriculum operations, document ingestion, verification, graph projection, review.	FR-13/14 services and graph worker.

The repository has detailed LLD documents but no exported diagram images were identified during report preparation. The descriptions above define what the diagrams should show; add the actual flowchart, UML and DFD figures if they are required as submitted evidence.

2.3 Design decisions

Decision	Reason	Alternative rejected
Layered API/service/repository	Separation of transport, business rules and persistence; easier to test and maintain.	Business logic in routers or repositories.
PostgreSQL authority; Neo4j projection	Relational transactions/version history remain authoritative; graph is rebuildable and optional.	Dual authoritative databases or requiring Neo4j for all operations.
Deterministic verification + human review	Repeatable checks, evidence codes and review status improve auditability.	LLM-generated authoritative accept/reject decisions.

2.4 Traceability

Req.	Design element
R1	JWT authentication, role dependencies and API security.
R2	Curriculum/version services, relational models and Alembic migrations.
R3	Outcome/mapping services, version workflows and review history.
R4	Document API/service/repository, storage adapter and extraction/LLM service.
R5	Prerequisite service, graph projection and graph/rule analysis.
R6	Version snapshot, verification checks, report and review APIs.
R7	Read-only analysis, evidence, governed references, fingerprints and human-review flag.

3. Implementation and functionality

3.1 Progress against checkpoints

The README labels the overall status “Initial Development” while listing several implemented feature areas. Week-specific completion percentages were not verifiable from repository artifacts, so this report does not present estimated percentages as facts.

Template checkpoint	Repository evidence	Status
Core implementation (Week 8; ~30%)	Authentication, curriculum CRUD/versioning, outcomes/mappings and prerequisite management are documented as completed.	Week/date/% require project records.
Refine/debug (Week 10; ~55%)	Document ingestion, knowledge graph projection and verification modules/tests exist.	Milestone result not independently verified.
Phase-1 consolidation (Week 14; ~70%)	Frontend workspace and API/LLD/test coverage span multiple modules.	Integration and completion must be confirmed by supervisor.

3.2 Major implemented features

Feature	Behavior and relevant source areas
Authentication	JWT/security and roles: `backend/app/services/auth.py`, `backend/app/core/security.py`, `backend/app/api/v1/auth.py`.
Curriculum management	Programs, curricula, versions, semesters, courses and version comparison: `services/curriculum.py`, repositories and Alembic migrations.
Outcomes and mapping	PO/PEO and CO–PO mapping/review: `services/outcomes.py`, `services/mappings.py`.
Prerequisites	Prerequisite workflow and verification: `services/prerequisites.py` and verification service.
Document processing	Upload, validation, extraction and LLM analysis: `services/documents.py`, `services/doc_extract.py`, `services/llm.py`.
Knowledge graph	Version-scoped projection, rebuild and worker: `services/knowledge_graph_projection.py`, `workers/graph_projection.py`.
Verification	Structure, prerequisites, topic overlap/progression, outcomes and governed references: `services/verification.py`, `services/verification_analysis.py`.
Frontend	Next.js App Router workspace, curriculum screens and knowledge map under `Frontend/src/app` and `Frontend/src/components`.

3.3 Code quality and optimization

Architecture follows API → service → repository responsibilities, DTO schemas separate API contracts from persistence models, and schema changes use Alembic. Verification builds a version snapshot and SHA-256 fingerprint so changed source data can mark prior runs stale. Graph projection uses durable jobs and retryable idempotent rebuilds. These documented patterns improve traceability/reliability; no measured speedup is claimed.

3.4 Deviations / scope clarifications

The original concept discusses semantic similarity and broad accreditation/industry comparisons. The implemented verification design describes deterministic lexical topic analysis and governed configurable references; semantic/embedding analysis and official accreditation certification are not claimed as completed. Neo4j is optional: relational application features remain available without it.

4. Testing, debugging and integration

4.1 Test cases

ID / type	Input scenario	Expected result	Actual / status
T1 positive	Valid version, courses, approved outcome mappings and valid prerequisite DAG.	Run completes and returns version-scoped summary/findings.	Test not run for this report.
T2 boundary	Semester gap or credit value at defined limit.	Correct rule-specific finding or no finding at valid boundary.	Not run here.
T3 negative	Self/cyclic prerequisite, invalid course reference or malformed/empty upload.	Stable validation/error/finding; no silent corruption.	Not run here.
T4 stale	Modify curriculum after saved run.	Fingerprint differs; old report marked stale.	Not run here.
T5 graph unavailable	Neo4j missing or down.	Relational features work; graph API signals unavailable/stale state.	Behavior specified; not executed.

4.2 Test summary

Test modules exist for authentication/security, curriculum, academic APIs, mappings, prerequisites, document API, graph worker/projection, verification analysis and review. No test or lint command was executed as part of report generation; total/pass/fail/coverage numbers are not available. Recommended commands: `cd backend && uv run pytest && uv run ruff check .`; `cd Frontend && npm run lint && npm run build`.

4.3 Bugs found and fixed

No verified defect/root-cause/fix evidence was supplied. This section should contain a real issue from test or integration history, its triggering input, root cause, fix and before/after output. No bug is fabricated here.

4.4 Integration of modules

The frontend calls authenticated API routes; routes delegate to services and repositories. PostgreSQL commits source changes and records graph projection jobs; the worker refreshes Neo4j independently. Verification reads a selected version snapshot, writes findings and review status to relational storage, and does not mutate curriculum data. Graph unavailability should not block relational operations. Specific integration bugs/fixes require actual logs.

5. Documentation, Phase-2 plan and viva readiness

5.1 Individual contribution

The codebase documents team-level modules but does not establish which commits or tasks belong to Vignesh V Hegde. Personal contribution statement intentionally requires your verified work history: list design, implementation and tests you personally completed, with dates/weeks and file/PR references. Do not claim work completed by teammates.

5.2 Phase-2 pending-work plan

Pending work	Priority	Approach	Target
Run quality gates and record outcomes	High	Execute backend pytest/Ruff and frontend lint/build; fix failures and attach logs.	Before review
Attach actual flowchart/UML/DFD	High	Create diagrams and map entities/processes to source paths.	Before viva
Validate analysis quality	High	Use representative curricula and faculty-labelled cases; measure false positives/negatives.	Phase 2
Benchmark scale/performance	Medium	Measure verification, document processing and graph projection on larger fixtures.	Phase 2
Evaluate semantic comparison	Medium	If approved, compare embedding approach to deterministic baseline on labelled data.	Phase 2
Operational readiness	Medium	Verify deployment config, secrets, migrations, worker retries, backups and monitoring.	Phase 2

5.3 Challenges and learning

A core design challenge is combining relational consistency with graph-oriented analysis. The documented choice keeps PostgreSQL authoritative and Neo4j rebuildable, with durable projection jobs and freshness status. Another challenge is keeping automated findings explainable: stable checks, evidence, severity/confidence and mandatory human review reduce the risk of treating a heuristic as an academic verdict. A personal debugging story requires the student’s actual experience and is not invented.

5.4 Viva readiness

Prepare to explain: why PostgreSQL is authoritative and Neo4j optional; how API/service/repository layers map to source; prerequisite graph cycle/path checks and verified complexity; the snapshot fingerprint and stale-run behavior; limits of lexical similarity and LLM extraction; an actual bug and test; personal contribution; and what remains for Phase 2. Quote complexity only after verifying the implementation.

Declaration

I declare that this report is my own work and that evidence submitted is genuine. Project descriptions are based on repository documentation. Personal contribution, milestones, test outcomes and signatures must be verified by the student.

Student signature / date	Faculty guide signature / date
_____	_____